STAFFSYNC-HRM

Full Stack Development Project Documentation

A practical HR management portal for everyday employee tasks

Project title	**StaffSync-HRM**
Technology stack	MongoDB, Express.js, React, Node.js
Prepared by	Razeena Shaikh
Demo URL	**staff-sync-hrm.vercel.app**

1. Introduction

StaffSync-HRM is a web application that brings common employee and HR activities into one place. Instead of handling every task through separate messages or manual records, employees can use the portal to view information and submit routine requests. The project was built as a full-stack application using the MERN stack.

The aim is to make everyday HR tasks easier to follow while keeping the interface straightforward for employees.

Project details

- Project title: StaffSync-HRM
- Project type: Human Resource Management web portal
- Frontend: React with Vite
- Backend: Node.js and Express.js
- Database: MongoDB
- Project Developer: Razeena Shaikh

2. Project Overview

StaffSync-HRM provides a shared portal for employee-related activities. The current interface includes an employee dashboard and sections for profile information, attendance, leave requests, announcements, and training. It is intended to reduce confusion around routine requests and make important information easier to find.

Main features

- Employee registration and login.
- Employee profile and dashboard views.
- Attendance check-in and attendance-related information.
- Leave request submission and status tracking.
- Announcements for workplace updates.
- Training information for employee learning activities.
- A central interface that groups these tasks together.

3. Architecture

The application follows a client-server structure. The React frontend handles pages and user interactions. The Express backend exposes API routes, processes requests, and communicates with MongoDB.

Frontend

The frontend is built with React and Vite. React components organise the interface into reusable parts, while Vite provides the development server and production build process. The client is located in the client directory.

Backend

The backend uses Node.js and Express.js to receive HTTP requests and return responses. Route files are organised by feature, including authentication, employees, leaves, attendance, announcements, and training. The server code is located in the server directory.

Database

MongoDB stores application records, and Mongoose is used by the backend to connect to the database and work with data models. The exact fields and validation rules should be confirmed from the models in the repository.

Request flow

1. A user opens a page in the React frontend.
2. The frontend sends a request to the backend API when data is needed.
3. Express handles the request through the relevant route and model.
4. The backend reads or updates MongoDB and returns a response.
5. The frontend displays the result to the user.

4. Setup Instructions

Prerequisites

- Node.js and npm.
- Git.
- MongoDB Atlas or a local MongoDB instance.
- Visual Studio Code or another code editor.

Repository: https://github.com/razeenaashaikh1217/StaffSync-HRM

Installation

1. Clone the repository or download the project from GitHub.
2. Open the project folder in Visual Studio Code.
3. Open a terminal in server and run npm install.
4. Create server/.env and add MONGO_URI=your_mongodb_connection_string.
5. Add any other environment variables required by the current authentication code, such as JWT_SECRET if used.
6. Open a second terminal in client and run npm install.
7. Check the frontend API configuration so it points to the correct backend URL.
8. Start both parts using the commands in Section 6.

Do not commit .env files or paste database credentials into a public repository. Keep deployment secrets in the hosting provider's environment-variable settings.

5. Folder Structure

The repository is split into two main parts.

Client

- client/ — React frontend application.
- client/src/ — pages, components, and client-side logic; exact subfolders may vary.
- client/package.json — frontend dependencies and scripts.

Server

- server/ — Node.js and Express backend.
- server/index.js — backend entry point and route registration.

- server/routes/ — route handlers for authentication, employees, leaves, attendance, announcements, and training.
- server/models/ — Mongoose models.
- server/middleware/ — shared middleware, including request protection where implemented.
- server/package.json — backend dependencies and scripts.

Use the actual folder names in the repository if they differ from this overview.

6. Running the Application

For local development, run the backend and frontend in separate terminal windows.

Backend

In the server directory, run:

```
npm install
npm start
```

The backend should print a message confirming that it started and connected to MongoDB. The local port is determined by the server configuration, commonly through the PORT environment variable.

Frontend

In the client directory, run:

```
npm install
npm run dev
```

Vite usually displays a local address such as http://localhost:5173. Open the address shown in your terminal.

Note: the reference document lists npm start for the frontend. For a standard Vite project, npm run dev is the usual development command; follow the scripts in client/package.json.

7. API Documentation

The backend is organised into feature-based route groups. Confirm exact paths, HTTP methods, request fields, and response formats from the current route files before final submission.

Route group	Purpose	Implementation file
/api/auth	Registration and login	server/routes/auth.js
/api/employees	Employee records and related operations	server/routes/employees.js
/api/leaves	Leave requests and status	server/routes/leaves.js
/api/attendance	Attendance operations	server/routes/attendance.js
/api/announcements	Workplace announcements	server/routes/announcements.js
/api/trainings	Training information	server/routes/trainings.js

Example response from the backend health route, if unchanged:

```
{ "message": "StaffSync HRM Backend is running!" }
```

For each endpoint, add its exact method (GET, POST, PUT/PATCH, DELETE), required parameters, authentication requirements, and an example response from the current code.

8. Authentication

The project includes registration and login functionality. The authentication route checks submitted credentials and returns an appropriate result. If the current implementation issues JSON Web Tokens (JWTs), the frontend should send the token with protected requests and the backend should verify it through authentication middleware.

Confirm the exact token behaviour in server/routes/auth.js and server/middleware/. If JWT is used, keep the signing secret in an environment variable and never store it in source control. Passwords should be stored as hashes rather than plain text.

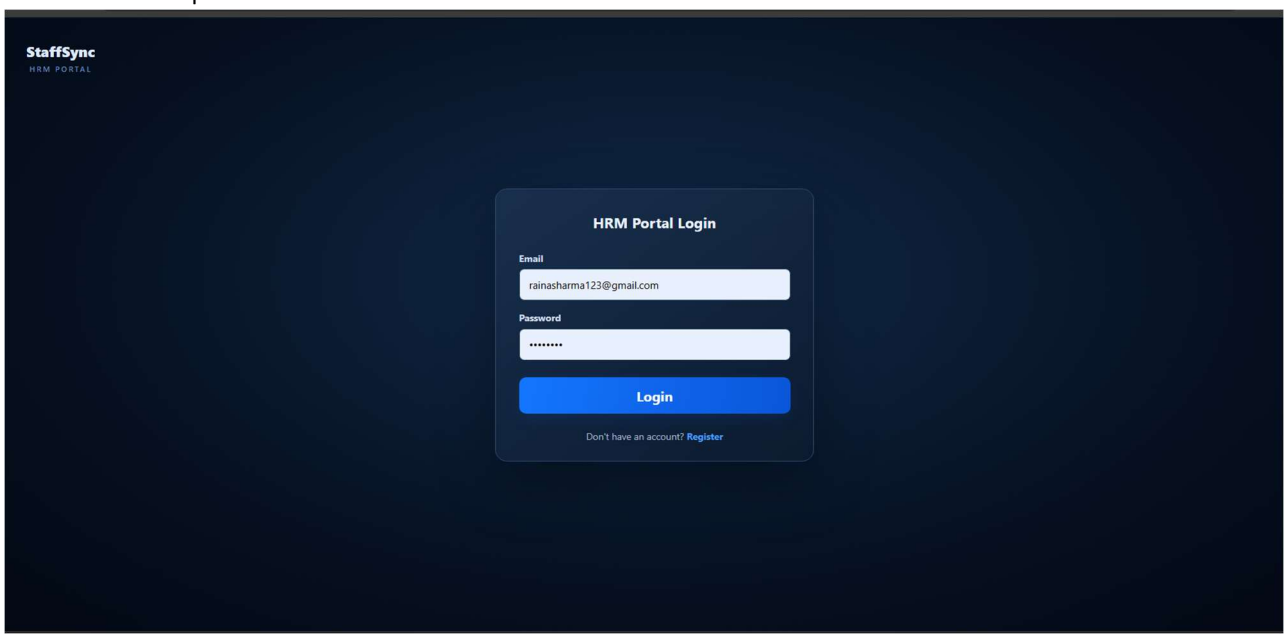
9. User Interface

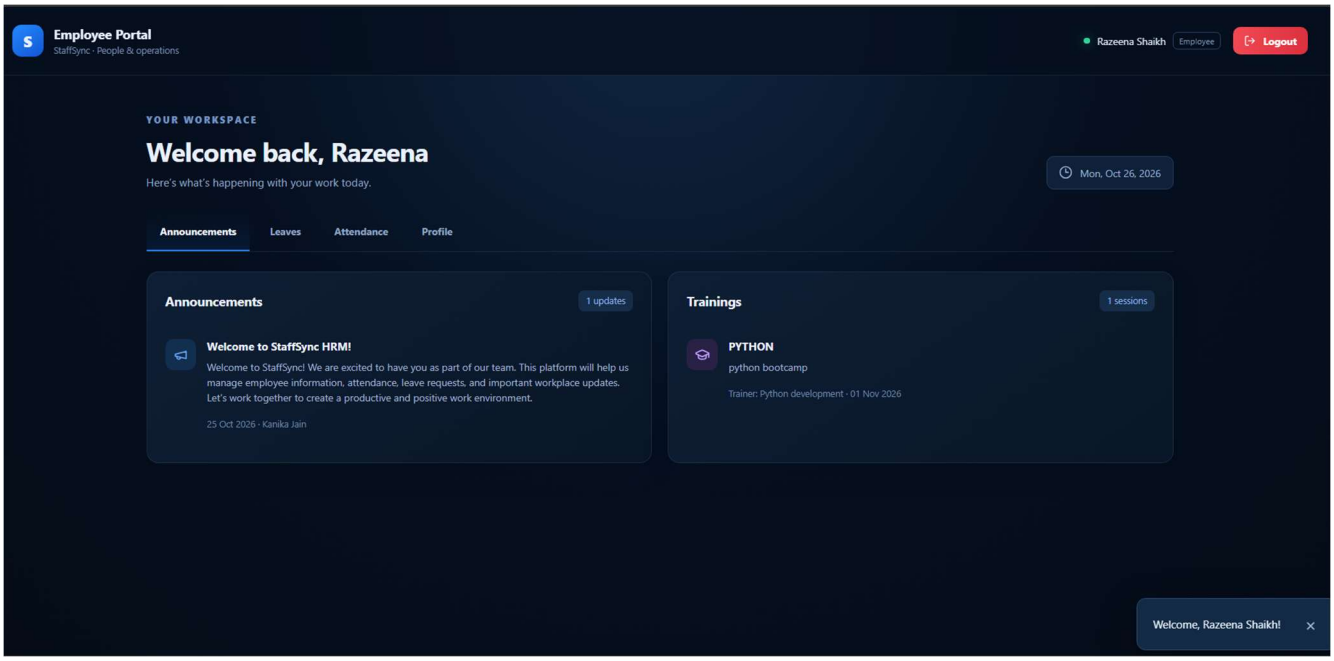
The interface is arranged around tasks an employee may need during a normal workday. The landing page introduces the portal, and the employee area provides access to the main features.

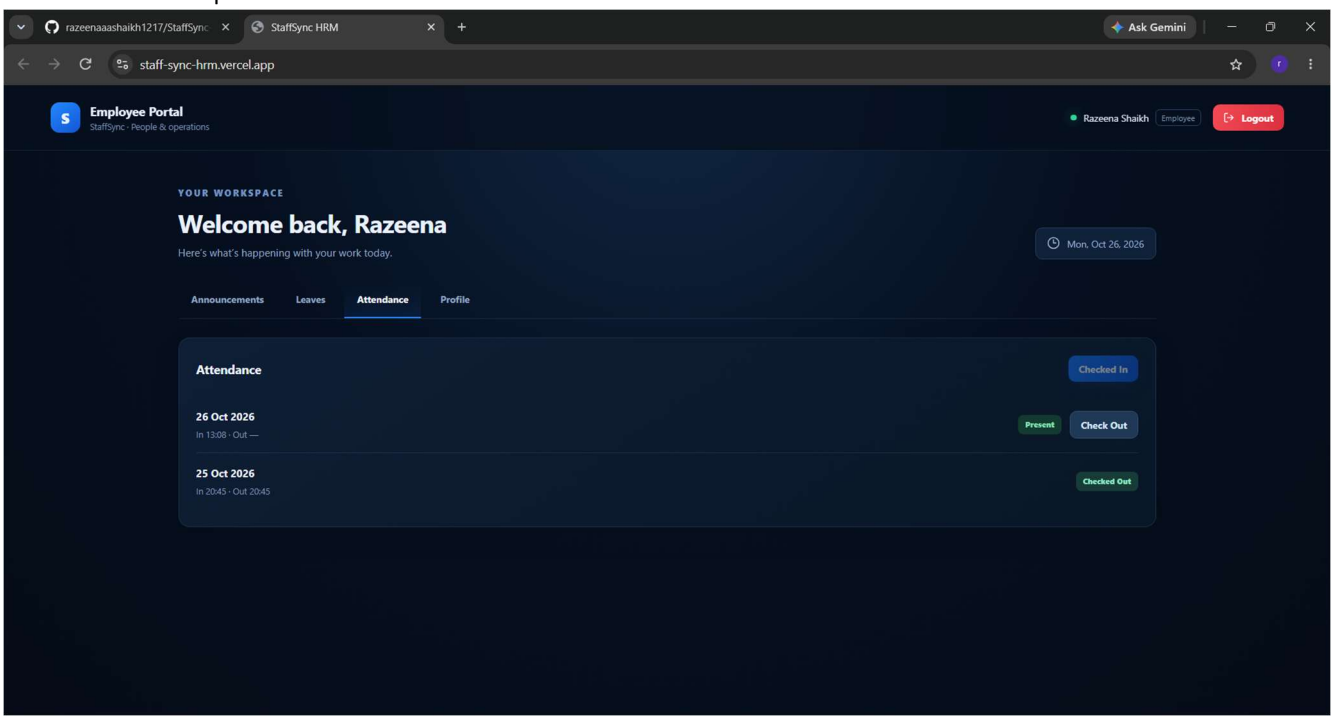
- Landing page: introduces StaffSync-HRM and its purpose.
- Login and registration: lets users access or create an account.
- Employee dashboard: central place to navigate the portal.
- Attendance: access to attendance actions.
- Leave requests: submit and review leave requests.
- Announcements and training: workplace updates and learning information.

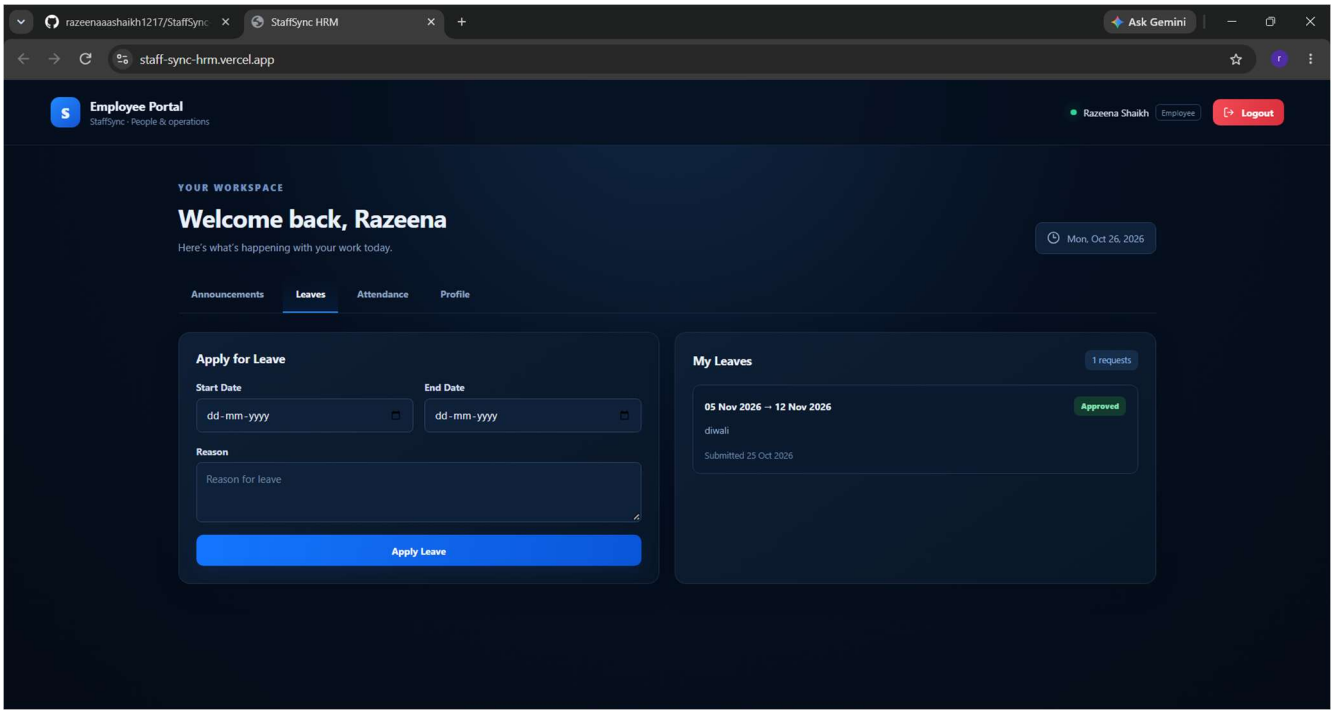
Add screenshots from your actual application in the spaces below.




StaffSync
HRM PORTAL
HRM Portal Login
Email
rainasharma123@gmail.com
Password
Login
Don't have an account? Register


Employee Portal
StaffSync · People & operations
Razeena Shaikh
Employee
Logout
YOUR WORKSPACE
Welcome back, Razeena
Here's what's happening with your work today.
Mon, Oct 26, 2026
Announcements
Leaves
Attendance
Profile
Announcements
1 updates
Welcome to StaffSync HRM!
Welcome to StaffSync! We are excited to have you as part of our team. This platform will help us manage employee information, attendance, leave requests, and important workplace updates. Let's work together to create a productive and positive work environment.
25 Oct 2026 · Kanika Jain
Trainings
1 sessions
PYTHON
python bootcamp
Trainer: Python development · 01 Nov 2026
Welcome, Razeena Shaikh!





10. Testing

Testing should cover the main user journeys and the connection between the frontend, backend, and database. Check successful actions as well as invalid input so that the application gives clear feedback.

- Open the landing page and confirm navigation works.
- Register a test account and verify the result.
- Log in with valid credentials and test an invalid password.
- Submit a leave request and check that it appears as expected.

- Test attendance actions and verify the result.
- Check announcements and training information.
- Review backend logs when an API request fails.
- After deployment, test the public frontend and every API route it uses.

Testing tools used: Google Chrome for browser-based UI testing and manual verification of application workflows. GitHub and browser developer tools were used for deployment checks and troubleshooting, where applicable.

11. Screenshots or Demo

Frontend URL:[**staff-sync-hrm.vercel.app**]

Backend API URL: [https://staffsync-hrm-9s08.onrender.com]

Before sharing the demo, test registration, login, and the main employee features online. A page loading successfully does not by itself confirm that the API and database are connected.

12. Known Issues

No known issues were identified during the final testing of the current version of StaffSync-HRM. Further testing may identify improvements for future releases.

13. Future Enhancements

The following improvements could make StaffSync-HRM more useful in a real workplace:

- Add role-based access for employees, HR staff, and administrators.
- Improve leave approval workflows and request status updates.
- Add attendance summaries and downloadable reports.
- Provide notifications for leave decisions and important announcements.
- Improve mobile responsiveness and accessibility.
- Add more validation, error handling, and automated tests.
- Add audit logs for important HR actions where appropriate.